

Embodied AI
Lab Manual for Isaac Lab/Isaac Sim
R1 Rough-Terrain Setup
August 2026

Prepared by

Mario Ortiz

Table of Contents

1	Objective.....	3
2	Preface about the Code	3
3	File Structure:	3
4	Official G1 Rough-Terrain Environment	4
5	R1 Porting.....	5
6	R1 Configuration Steps/Changes.....	5
6.1	Main Robot Asset Change	5
6.2	Height Scanner Change	6
6.3	Terrain Modification	6
6.4	Reward Function Changes	6
6.4.1	Feet Air Time	7
6.4.2	Anti-Hopping Penalty.....	7
6.4.3	Swing Knee Flexion Penalty	7
6.5	Joint Deviation Changes.....	8
6.6	Command Range Changes	8
6.7	Termination Condition Changes	9
6.8	Setting up the R1 Rough-Terrain Training Pipeline	9
6.8.1	Creating R1 Folder.....	10
6.8.2	Adding the R1 Asset Configuration File.....	10
6.8.3	Adding the RSL-RL PPO Configuration.....	10
6.8.4	Registering the R1 Rough Task	12
6.8.5	Training Setup Verification.....	13
6.9	Running the R1 Rough-Terrain Training Pipeline.....	13

1 Objective

The goal of this lab is to showcase to students how a rough-terrain locomotion task can be adapted from the official **Unitree G1 Isaac Lab configuration** to a custom **Unitree R1 Configuration**.

In the previous lab, the Unitree R1 URDF file was imported into Isaac Sim and converted into a USD robot asset. This USD asset allows the R1 robot model to be used within Isaac Sim and Isaac Lab. However, creating the robot asset is only the first step. To train the R1 using reinforcement learning, Isaac Lab also requires an environment configuration that defines how the robot interacts with the simulation.

This configuration specifies key components of the training environment, including the robot asset, sensors, observations, actions, commands, rewards, terrains, events and termination conditions.

In this lab, we will examine how the **rough_env_cfg.py** file for the R1 was created by adapting the official G1 rough-terrain locomotion environment. By modifying the existing G1 configuration rather than building an environment from scratch, we can reuse much of the established locomotion framework while replacing and adjusting the components necessary for the R1.

2 Preface about the Code

This manual is not intended to have the student write the entire code from scratch. Instead, the purpose of this lab is for the student to understand how the provided code works by reviewing its structure, major configuration changes, and the reasoning behind each section.

The necessary code files are provided as downloadable files for this lab. These files should be copied into the appropriate Isaac Lab directory described in the manual.

Also, for the task to work correctly, the environment configuration file must be placed in the correct configuration folder. In addition, the task-specific classes from that file must be referenced in the proper support scripts. For example, if the task uses **rough_env_cfg.py**, then the corresponding classes such as **R1RoughEnvCfg** must be imported and registered in **__init__.py**.

Similarly, the task-specific PPO runner class must be added to **rsl_rl_ppo_cfg.py**. For example, a rough-terrain R1 task will require the **R1RoughPPORunnerCfg** to be added to the file.

3 File Structure:

C:\isaac\lab

```

├─ source
|   └─ isaac\lab_tasks
|       └─ isaac\lab_tasks
|           └─ manager_based
|               └─ locomotion

```

```

|           └─ velocity
|           └─ config
|           └─ r1
|               ├── __init__.py
|               ├── r1_asset_cfg.py
|               ├── rough_env_cfg.py
|               |
|               ├── assets
|               |   └─ R1
|               |       ├── R1.usd
|               |       └─ configuration
|               |
|               └─ agents
|                   ├── __init__.py
|                   └─ rsl_r1_ppo_cfg.py
|
└─ logs
    └─ rsl_r1
        └─ r1_rough
            └─ YYYY-MM-DD_HH-MM-SS
                ├── model_0.pt
                ├── model_3500.pt
                └─ model_6999.pt

```

4 Official G1 Rough-Terrain Environment

The Unitree G1 has an official Isaac Lab rough-terrain locomotion environment, making its configuration a useful reference for understanding the structure of a locomotion task. The G1 rough-terrain environment is built on top of Isaac Lab's standard rough-terrain locomotion template:

```
class G1RoughEnvCfg(LocomotionVelocityRoughEnvCfg):
```

For the R1, the corresponding configuration uses the same parent class:

```
class R1RoughEnvCfg(LocomotionVelocityRoughEnvCfg):
```

This means the R1 environment does not need to be built entirely from scratch. Instead, it inherits the existing rough-terrain locomotion framework provided by Isaac Lab. The primary task is to modify the robot-specific components of the environment so that they accurately reflect the R1's structure, joints, sensors, and physical properties rather than those of the G1.

5 R1 Porting

Although both environments inherit from **LocomotionVelocityRoughEnvCfg**, the R1 cannot use the G1 configuration without modification.

Isaac Lab environments reference specific body names, joint names, sensors, and contact links defined by the robot asset. These references must match the corresponding names in the robot's USD file. Because the G1 and R1 have different robot structures and naming conventions, the G1-specific body and joint references cannot be used directly for the R1.

For example, the G1 configuration uses:

```
torso_link
```

as its primary body link, while the R1 configuration uses:

```
pelvis_link
```

instead. In the R1 code, the height scanner is attached to `pelvis_link` rather than `torso_link`. Therefore, the R1 environment must be modified so that the Isaac Lab's rewards, sensors, events, and terminations reference the correct R1 links and joints.

6 R1 Configuration Steps/Changes

6.1 Main Robot Asset Change

In the G1 file, the robot is assigned using NVIDIA's built-in G1 asset configuration:

```
self.scene.robot = G1_MINIMAL_CFG.replace(
    prim_path="{ENV_REX_NS}/Robot"
)
```

For the R1 version, this is replaced with the custom R1 asset configuration:

```
self.scene.robot = R1_EDU_CFG.replace(
    prim_path="{ENV_REGEX_NS}/Robot"
)
```

The R1 file imports this custom asset configuration from **r1_asset_cfg.py**.

The USD file must be wrapped inside an Isaac Lab articulation configuration, such as `R1_EDU_CFG`, so Isaac Lab knows how to spawn and simulate the robot.

6.2 Height Scanner Change

The rough-terrain locomotion task uses a height scanner to provide the policy with information about the surrounding terrain, allowing the policy to account for changes in terrain height when learning to walk over uneven surfaces.

For the G1, the height scanner is attached to the reference torso_link. For the R1, this reference changes to **pelvis_link**:

```
self.scene.height_scanner.prim_path = (
    "{ENV_REGEX_NS}/Robot/pelvis_link"
)
```

The height scanner must be attached to an appropriate body link on the R1 so that terrain measurements are taken relative to the correct part of the robot. Using the correct link ensures that the resulting terrain observations accurately represent the environment from the robot's perspective.

6.3 Terrain Modification

The original G1 rough_terrain environment uses Isaac Lab's inherited rough-terrain generator, which includes multiple terrain types. For the R1 configuration, we simplify the terrain setup by retaining only the **random_rough** terrain type.

The R1 configuration introduces a helper function named:

```
keep_only_random_rough_terrain(...)
```

This function removes the other terrain types and configures the terrain layout by adjusting the number of rows, number of columns, curriculum settings, and tile size.

The R1 training configuration uses:

```
keep_only_random_rough_terrain(
    self.scene.terrain,
    num_rows=10,
    num_cols=20,
    curriculum=True,
    tile_size=(8.0, 8.0),
)
```

6.4 Reward Function Changes

The G1 task already includes several common locomotion reward terms, including velocity tracking, yaw tracking, foot air time, foot sliding penalties, joint limit penalties, and joint deviation penalties.

The R1 task maintains the same general reward structure but modifies several reward terms to encourage more realistic and stable walking behavior.

6.4.1 Feet Air Time

The G1 task uses the ankle roll links to identify the robot's feet. The R1 task follows the same approach but explicitly specifies the left and right R1 ankle links:

```
body_names=[
    "left_ankle_roll_link",
    "right_ankle_roll_link",
]
```

The R1 configuration also lowers the foot air-time threshold the G1 value of 0.4 to 0.25. This encourages more frequent stepping while reducing excessive time spent with the feet off the ground.

6.4.2 Anti-Hopping Penalty

To discourage undesirable hopping behavior, the R1 configuration introduces a custom reward term called:

```
both_feet_airborne
```

This penalty is activated when both feet are off the ground while the robot is commanded to move. The reward term is configured as:

```
both_feet_airborne = RewTerm(
    func=both_feet_airborne_penalty,
    weight=-5.0,
    ...
)
```

By penalizing this behavior, the policy is encouraged to maintain at least one foot on the ground during locomotion, promoting a walking or stepping motion rather than jumping.

6.4.3 Swing Knee Flexion Penalty

The R1 configuration also introduces a reward term called:

```
swing_knee_flexion
```

This term penalizes the robot when its airborne leg does not bend the knee sufficiently during single-support walking. The goal is to encourage a more natural and effective stepping motion rather than stiff-legged movement.

The reward term references the R1's knee joints:

```
joint_names=[
    "left_knee_joint",
    "right_knee_joint",
]
```

6.5 Joint Deviation Changes

The G1 robot has several joint groups that do not directly apply to the R1. For example, the G1 configuration includes penalties for finer joint deviation. Because the R1 does not use the same finger joint structure, the R1 configuration removes this reward term and replaces the joint groups with relevant ones to the R1.

The R1 configuration includes separate penalties for:

Hip roll joints

Hip yaw joints

Waist joints

Arm joints

Head joints

These penalties help discourage unstable or unnecessary movement in joints that are not primarily responsible for locomotion.

6.6 Command Range Changes

The G1 rough-terrain task allows a forward velocity command in the range:

```
lin_vel_x = (0.0, 1.0)
```

For the R1, the forward velocity range is reduced to a more moderate value:

```
self.commands.base_velocity.ranges.lin_vel_x = (
    0.0,
    0.7,
)
```

The yaw velocity command remains unchanged:

```
self.commands.base_velocity.ranges.ang_vel_z = (
    -1.0,
    1.0,
)
```

This allows the R1 to continue learning to respond to turning commands while limiting the maximum forward speed during the early stages of gait development

6.7 Termination Condition Changes

The G1 configuration terminates an episode when the torso makes contact with the ground:

```
self.terminations.base_contact.params["sensor_cfg"].body_names = "torso_link"
```

For the R1, monitoring a single body link is not sufficient because the robot could fall or make undesirable contact with other parts of its body without the pelvis_link necessarily triggering the termination conditions. Therefore, the R1 configuration expands the list of monitored body parts.

The R1 termination condition checks for contact involving:

```
pelvis
waist
head
hip links
knee links
shoulder links
elbow links
wrist links
```

This makes fall detection stricter and more appropriate for the R1 body structure.

6.8 Setting up the R1 Rough-Terrain Training Pipeline

After modifying the rough_env_cfg.py file, the next step is to verify that the R1 environment can successfully launch, train, save checkpoints, and play trained policies.

Before training can begin, the custom R1 task files must be added to the Isaac Lab task configuration directory. The official task files are located at:

```
C:\isaac\source\isaac\isaac_tasks\isaac_tasks\manager_based\locomotion\velocity\config\g1
```

The custom R1 task files should follow the same general structure and be placed in:

```
C:\isaac\source\isaac\isaac_tasks\isaac_tasks\manager_based\locomotion\velocity\config\r1
```

The resulting folder structure should be:

```
r1
├─ __init__.py
├─ r1_asset_cfg.py
├─ rough_env_cfg.py
└─ agents
    ├─ __init__.py
    └─ rsl_r1_ppo_cfg.py
```

Each file serves a specific purpose:

- `r1_asset_cfg.py`: points Isaac Lab to the exported R1 USD asset and defines the R1 robot configuration
- `rough_env_cfg.py`: defines the R1 rough-terrain locomotion environment
- `rsl_rl_ppo_cfg.py`: defines the PPO training configuration used by RSL-RL.
- `__init__.py`: registers the R1 task names so that Isaac Lab can recognize and launch the custom environments.

6.8.1 Creating R1 Folder

Open PowerShell and run:

```
cd C:\isaac\lab\source\isaac\lab_tasks\isaac\lab_tasks\manager_based\locomotion\velocity\config
```

Create the R1 folder and its agents subdirectory

```
mkdir r1
```

```
mkdir r1\agents
```

Next, create the required Python files:

```
New-Item -ItemType File -Force r1\__init__.py
```

```
New-Item -ItemType File -Force r1\rough_env_cfg.py
```

```
New-Item -ItemType File -Force r1\r1_asset_cfg.py
```

```
New-Item -ItemType File -Force r1\agents\__init__.py
```

```
New-Item -ItemType File -Force r1\agents\rsl_rl_ppo_cfg.py
```

Open the files in Visual Studio Code (or another source-code editor).

Example (in the `r1` folder):

```
code rough_env_cfg.py
```

6.8.2 Adding the R1 Asset Configuration File

Open the R1 asset configuration file:

```
code r1\r1_asset_cfg.py
```

This file defines the R1 robot asset and specified the location of the exported R1 USD file.

Copy the provided R1 asset configuration code into this file. Next, add the previously developed R1 rough-terrain environment code to `r1\rough_env_cfg.py`

6.8.3 Adding the RSL-RL PPO Configuration

The R1 PPO configuration is based on the official G1 configuration. The overall training structure remains similar, but the class and experiment names are changed to identify the R1 task.

```
from isaacgym.utils import configclass

from isaacgym_r1.r1 import (
    R1OnPolicyRunnerCfg,
    R1PpoActorCriticCfg,
    R1PpoAlgorithmCfg,
)

@configclass
class R1RoughPPORunnerCfg(R1OnPolicyRunnerCfg):
    num_steps_per_env = 24
    max_iterations = 7000
    save_interval = 50
    experiment_name = "r1_rough"

    policy = R1PpoActorCriticCfg(
        init_noise_std=1.0,
        actor_obs_normalization=False,
        critic_obs_normalization=False,
        actor_hidden_dims=[512, 256, 128],
        critic_hidden_dims=[512, 256, 128],
        activation="elu",
    )

    algorithm = R1PpoAlgorithmCfg(
        value_loss_coef=1.0,
        use_clipped_value_loss=True,
        clip_param=0.2,
        entropy_coef=0.008,
        num_learning_epochs=5,
        num_mini_batches=4,
        learning_rate=1.0e-3,
        schedule="adaptive",
        gamma=0.99,
```

```

        lam=0.95,
        desired_kl=0.01,
        max_grad_norm=1.0,
    )

```

6.8.4 Registering the R1 Rough Task

Open the task registration file:

```
code r1\__init__.py
```

The R1 rough-terrain training and play environments must be registered with Gymnasium, following the same approach used by the G1.

Add:

```

import gymnasium as gym
from . import agents
from .rough_env_cfg import R1RoughEnvCfg, R1RoughEnvCfg_PLAY

```

```

gym.register(
    id="Isaac-Velocity-Rough-R1-v0",
    entry_point="isaacclab.envs:ManagerBasedRLEnv",
    disable_env_checker=True,
    kwargs={
        "env_cfg_entry_point": R1RoughEnvCfg,
        "rsl_rl_cfg_entry_point": f"{agents.__name__}.rsl_rl_ppo_cfg:R1RoughPPORunnerCfg",
    },
)

```

```

gym.register(
    id="Isaac-Velocity-Rough-R1-Play-v0",
    entry_point="isaacclab.envs:ManagerBasedRLEnv",
    disable_env_checker=True,
    kwargs={
        "env_cfg_entry_point": R1RoughEnvCfg_PLAY,
        "rsl_rl_cfg_entry_point": f"{agents.__name__}.rsl_rl_ppo_cfg:R1RoughPPORunnerCfg",
    },
)

```

```
    },
)
```

This creates two task names:

Isaac-Velocity-Rough-R1-v0

and:

Isaac-Velocity-Rough-R1-Play-v0

The first task is used for training, while the second is used for playing and evaluating trained policies.

6.8.5 Training Setup Verification

After creating and configuring the files, run a short training session to verify that Isaac Lab recognizes the new R1 task.

```
cd C:\isaacLab
.\isaacLab.bat -p .\scripts\reinforcement_learning\rsl_rl\train.py `
  --task Isaac-Velocity-Rough-R1-v0 `
  --num_envs 4 `
  --max_iterations 1
```

If the task is registered correctly, Isaac Lab should begin setting up the R1 rough-terrain environment.

6.9 Running the R1 Rough-Terrain Training Pipeline

For this experiment, the R1 rough-terrain environment will be trained for 7,000 iterations. This provides sufficient training time for the policy to develop and improve its locomotion behavior.

Run the following command to begin training:

```
cd C:\isaacLab
.\isaacLab.bat -p .\scripts\reinforcement_learning\rsl_rl\train.py `
  --task Isaac-Velocity-Rough-R1-v0 `
  --num_envs 1024 `
  --headless
```

Note that **\$r1RoughPath** represents the specific folder in which your model is played in. You will find this name in the folder:

```
\isaacLab\logs\rsl_rl\
```

Run this command to play the 3,500th iteration run:

```
cd C:\isaacrab
.\isaacrab.bat -p .\scripts\reinforcement_learning\rsl_rl\play.py `
  --task Isaac-Velocity-Rough-R1-Play-v0 `
  --num_envs 32 `
  --checkpoint "$r1RoughPath\model_3500.pt"
```

Run this command to play the final iteration run:

```
cd C:\isaacrab
.\isaacrab.bat -p .\scripts\reinforcement_learning\rsl_rl\play.py `
  --task Isaac-Velocity-Rough-R1-Play-v0 `
  --num_envs 32 `
  --checkpoint "$r1RoughPath\model_6999.pt"
```

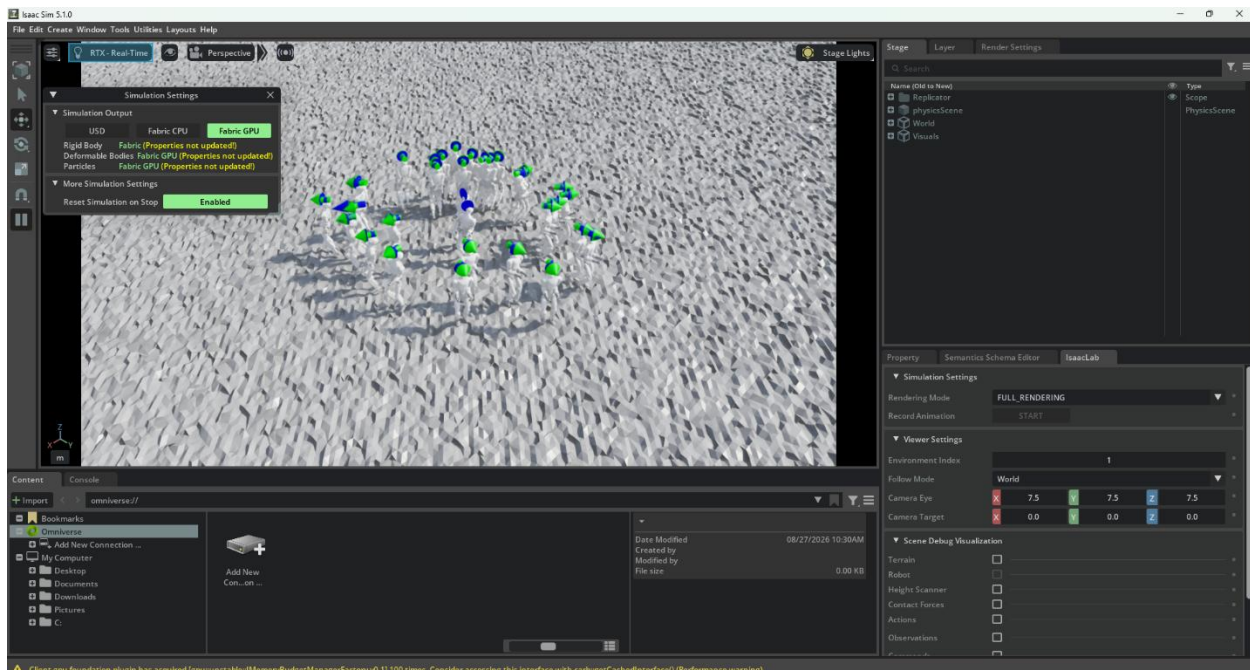


Figure 1: Initial Final Checkpoint Showcase of the Unitree R1 Rough-Environment Task

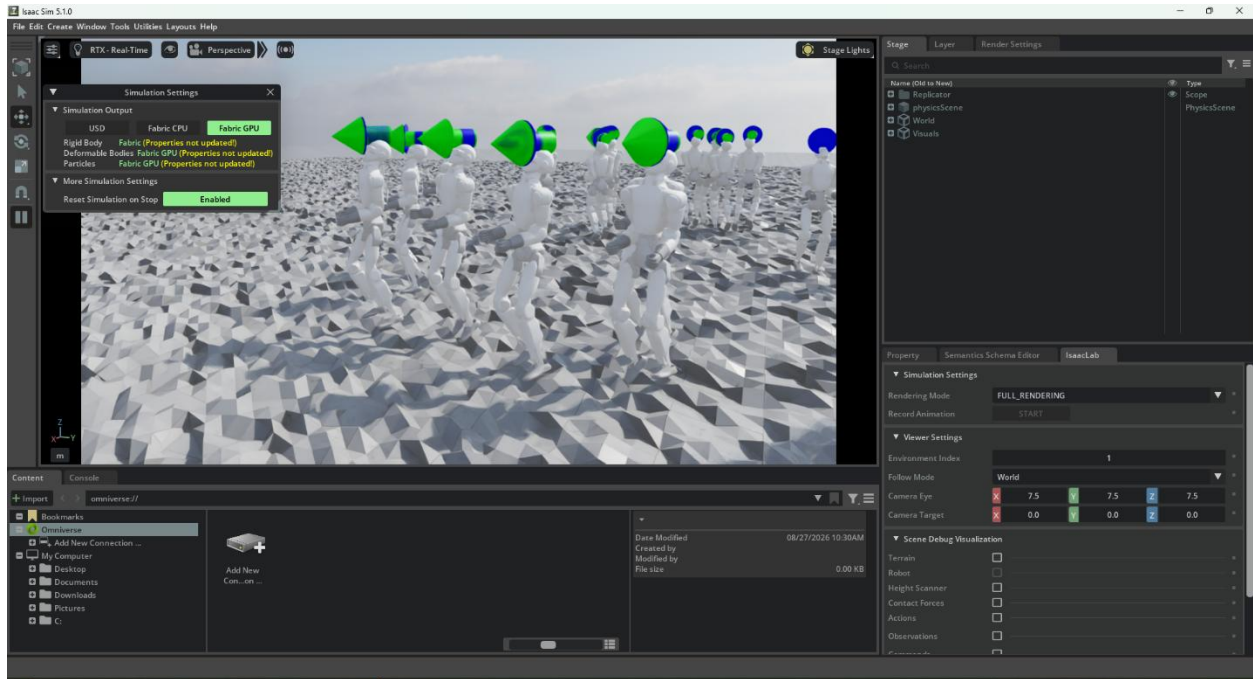


Figure 2: Closeup of Unitree R1 Rough-Environment Task (Note that the head of the robot is aiming upwards, due to the lack of rewards/penalties done for the neck joints that the R1 has → The Unitree R1 has a 2-DOF neck joint).

7 References

- [1] NVIDIA, “Creating a Manager-Based RL Environment,” Isaac Lab Documentation.
- [2] NVIDIA, “isaacsim.managers,” Isaac Lab API Documentation.
- [3] NVIDIA, “Registering an Environment,” Isaac Lab Documentation.
- [4] NVIDIA, “isaacsim_rl,” Isaac Lab RL API Documentation.
- [5] RSL-RL, “Configuration,” RSL-RL Documentation.
- [6] PyTorch, “torch,” PyTorch Documentation.